

Regular Expressions in Python

Tushar B. Kute,
<http://tusharkute.com>



Regular Expression

- A regular expression is a special sequence of characters that helps you match or find other strings or sets of strings, using a specialized syntax held in a pattern.
- Regular expressions are widely used in UNIX world.
- The Python module `re` provides full support for Perl-like regular expressions in Python.
- The `re` module raises the exception `re.error` if an error occurs while compiling or using a regular expression.

Regular Expression

- A Regular Expression (RegEx) is a sequence of characters that defines a search pattern. For example,

`^a...s$`

- The above code defines a RegEx pattern. The pattern is: any five letter string starting with a and ending with s.

Regular Expression

- A pattern defined using RegEx can be used to match against a string.

Expression	String	Matched?
<code>^a...s\$</code>	abs	No match
	alias	Match
	abyss	Match
	Alias	No match
	An abacus	No match

Example:

```
import re

pattern = '^a...s$'
test_string = 'abyss'
result = re.match(pattern, test_string)

if result:
    print("Search successful.")
else:
    print("Search unsuccessful.")
```

re.search()

- The `re.search()` method takes two arguments: a pattern and a string. The method looks for the first location where the RegEx pattern produces a match with the string.
 - If the search is successful, `re.search()` returns a match object; if not, it returns `None`.
- ```
match = re.search(pattern, str)
```

# Metacharacters

- Metacharacters are characters that are interpreted in a special way by a RegEx engine. Here's a list of metacharacters:

[ ] . ^ \$ \* + ? { } ( ) \ |

# Square Brackets

- Square brackets specifies a set of characters you wish to match.

| Expression | String    | Matched?  |
|------------|-----------|-----------|
| [abc]      | a         | 1 match   |
|            | ac        | 2 matches |
|            | Hey Jude  | No match  |
|            | abc de ca | 5 matches |



# Square Brackets

- Here, [abc] will match if the string you are trying to match contains any of the a, b or c.
- You can also specify a range of characters using - inside square brackets.

[a-e] is the same as [abcde].

[1-4] is the same as [1234].

[0-39] is the same as [01239].

# Square Brackets

- You can complement (invert) the character set by using caret ^ symbol at the start of a square-bracket.

[^abc] means any character except a or b or c.

[^0-9] means any non-digit character.

# Period

- A period matches any single character (except newline '\n').

| Expression | String | Matched?                          |
|------------|--------|-----------------------------------|
| ..         | a      | No match                          |
|            | ac     | 1 match                           |
|            | acd    | 1 match                           |
|            | acde   | 2 matches (contains 4 characters) |

# Caret

- The caret symbol ^ is used to check if a string starts with a certain character.

| Expression       | String           | Matched?                                                                  |
|------------------|------------------|---------------------------------------------------------------------------|
| <code>^a</code>  | <code>a</code>   | 1 match                                                                   |
|                  | <code>abc</code> | 1 match                                                                   |
|                  | <code>bac</code> | No match                                                                  |
| <code>^ab</code> | <code>abc</code> | 1 match                                                                   |
|                  | <code>acb</code> | No match (starts with <code>a</code> but not followed by <code>b</code> ) |

# Dollar

- The dollar symbol \$ is used to check if a string ends with a certain character.

| Expression | String  | Matched? |
|------------|---------|----------|
| a\$        | a       | 1 match  |
|            | formula | 1 match  |
|            | cab     | No match |

# Star

- The star symbol `*` matches zero or more occurrences of the pattern left to it.

| Expression | String | Matched?                          |
|------------|--------|-----------------------------------|
| ma*n       | mn     | 1 match                           |
|            | man    | 1 match                           |
|            | maaan  | 1 match                           |
|            | main   | No match (a is not followed by n) |
|            | woman  | 1 match                           |

# Plus

- The plus symbol + matches one or more occurrences of the pattern left to it.

| Expression | String | Matched?                          |
|------------|--------|-----------------------------------|
| ma+n       | mn     | No match (no a character)         |
|            | man    | 1 match                           |
|            | maaan  | 1 match                           |
|            | main   | No match (a is not followed by n) |
|            | woman  | 1 match                           |

- The question mark symbol ? matches zero or one occurrence of the pattern left to it.

| Expression | String | Matched?                             |
|------------|--------|--------------------------------------|
| ma?n       | mn     | 1 match                              |
|            | man    | 1 match                              |
|            | maaan  | No match (more than one a character) |
|            | main   | No match (a is not followed by n)    |
|            | woman  | 1 match                              |



# Braces

- Consider this code:  $\{n,m\}$ . This means at least  $n$ , and at most  $m$  repetitions of the pattern left to it.

| Expression | String      | Matched?                                       |
|------------|-------------|------------------------------------------------|
| a{2,3}     | abc dat     | No match                                       |
|            | abc daat    | 1 match (at <u>daa</u> t )                     |
|            | aabc daaat  | 2 matches (at <u>aabc</u> and <u>daaa</u> t )  |
|            | aabc daaaat | 2 matches (at <u>aabc</u> and <u>daaaa</u> t ) |

# Braces

- Let's try one more example. This RegEx `[0-9]{2,4}` matches at least 2 digits but not more than 4 digits

| Expression              | String        | Matched?                                          |
|-------------------------|---------------|---------------------------------------------------|
| <code>[0-9]{2,4}</code> | ab123csde     | 1 match (match at <u>ab123</u> csde )             |
|                         | 12 and 345673 | 3 matches ( <u>12</u> , <u>3456</u> , <u>73</u> ) |
|                         | 1 and 2       | No match                                          |

# Alteration

- Vertical bar | is used for alternation (or operator).

| Expression | String | Matched?                                                              |
|------------|--------|-----------------------------------------------------------------------|
| a b        | cde    | No match                                                              |
|            | ade    | 1 match (match at <u>a</u> de )                                       |
|            | acdbea | 3 matches (at <u>a</u> <u>c</u> <u>d</u> <u>b</u> <u>e</u> <u>a</u> ) |

- Here, a|b match any string that contains either a or b

# Group

- Parentheses () is used to group sub-patterns. For example, (a|b|c)xz match any string that matches either a or b or c followed by xz

| Expression | String | Matched?                                       |
|------------|--------|------------------------------------------------|
| a   b      | cde    | No match                                       |
|            | ade    | 1 match (match at <u>a</u> de )                |
|            | acdbea | 3 matches (at <u>a</u> <u>c</u> <u>d</u> bea ) |

# Backslash

- Backslash \ is used to escape various characters including all metacharacters. For example,
- \\$a match if a string contains \$ followed by a. Here, \$ is not interpreted by a RegEx engine in a special way.
- If you are unsure if a character has special meaning or not, you can put \ in front of it.
- This makes sure the character is not treated in a special way.

# Special Sequences

- Special sequences make commonly used patterns easier to write. Here's a list of special sequences:
  - \A
  - \B
  - \b
  - \d
  - \D
  - \S
  - \s
  - \w

- \A - Matches if the specified characters are at the start of a string.

| Expression | String     | Matched? |
|------------|------------|----------|
| \Athe      | the sun    | Match    |
|            | In the sun | No match |

- \b - Matches if the specified characters are at the beginning or end of a word.

| Expression | String        | Matched? |
|------------|---------------|----------|
| \bfoo      | football      | Match    |
|            | a football    | Match    |
|            | afootball     | No match |
| foo\b      | the foo       | Match    |
|            | the afoo test | Match    |
|            | the afootest  | No match |



- \B - Opposite of \b. Matches if the specified characters are not at the beginning or end of a word.

| Expression | String        | Matched? |
|------------|---------------|----------|
| \Bfoo      | football      | No match |
|            | a football    | No match |
|            | afootball     | Match    |
| foo\b      | the foo       | No match |
|            | the afoo test | No match |
|            | the afootest  | Match    |

# \d

- \d - Matches any decimal digit. Equivalent to [0-9]

| Expression | String | Matched?                                   |
|------------|--------|--------------------------------------------|
| \d         | 12abc3 | 3 matches (at <u>1</u> <u>2</u> <u>3</u> ) |
|            | Python | No match                                   |

# \d

- \D - Matches any non-decimal digit. Equivalent to `[^0-9]`

| Expression      | String   | Matched?                          |
|-----------------|----------|-----------------------------------|
| <code>\D</code> | 1ab34"50 | 3 matches (at 1 <u>a</u> b34"50 ) |
|                 | 1345     | No match                          |

# \s

- \s - Matches where a string contains any whitespace character. Equivalent to [ \t\n\r\f\v].

| Expression | String       | Matched? |
|------------|--------------|----------|
| \s         | Python RegEx | 1 match  |
|            | PythonRegEx  | No match |

- \S - Matches where a string contains any non-whitespace character. Equivalent to `[^ \t\n\r\f\v]`.

| Expression                      | String                           | Matched?                                                                      |
|---------------------------------|----------------------------------|-------------------------------------------------------------------------------|
| <input type="text" value="\S"/> | <input type="text" value="a b"/> | 2 matches (at <input type="text" value="a"/> <input type="text" value="b"/> ) |
|                                 | <input type="text"/>             | No match                                                                      |

# \w

- \w - Matches any alphanumeric character (digits and alphabets). Equivalent to [a-zA-Z0-9\_]. By the way, underscore \_ is also considered an alphanumeric character.

| Expression | String     | Matched?                   |
|------------|------------|----------------------------|
| \w         | 12&" : ; c | 3 matches (at 12&" : ; c ) |
|            | % "> !     | No match                   |

- \W - Matches any non-alphanumeric character.  
Equivalent to [^a-zA-Z0-9\_]

| Expression | String | Matched?                              |
|------------|--------|---------------------------------------|
| \W         | 1a2%c  | 1 match (at 1 <u>a</u> 2 <u>%</u> c ) |
|            | Python | No match                              |

- \Z - Matches if the specified characters are at the end of a string.

| Expression | String                    | Matched? |
|------------|---------------------------|----------|
| Python\Z   | I like Python             | 1 match  |
|            | I like Python Programming | No match |
|            | Python is fun.            | No match |



# re.findall()

- The re.findall() method returns a list of strings containing all matches.

# re.split()

- The re.split method splits the string where there is a match and returns a list of strings where the splits have occurred.
- If the pattern is not found, re.split() returns a list containing the original string.

# re.sub()

- The syntax of re.sub() is:  
`re.sub(pattern, replace, string)`
- The method returns a string where matched occurrences are replaced with the content of replace variable.
- If the pattern is not found, re.sub() returns the original string.

# re.subn()

- The `re.subn()` is similar to `re.sub()` except it returns a tuple of 2 items containing the new string and the number of substitutions made.

# Reference

- <https://www.programiz.com/>

# Thank you

*This presentation is created using LibreOffice Impress 5.1.6.2, can be used freely as per GNU General Public License*



@mitu\_skillologies



/mITuSkillologies



@mitu\_group



/company/mitu-  
skillologies



MITUSkillologies

## Web Resources

<https://mitu.co.in>  
<http://tusharkute.com>

[contact@mitu.co.in](mailto:contact@mitu.co.in)  
[tushar@tusharkute.com](mailto:tushar@tusharkute.com)